Máquinas de vectores de soporte (SVM)

Sesión 10 · Módulo 3: Modelos de clasificación

Margen máximo y problema dual · Margen blando y C · Truco del kernel · SVM multiclase

Dígitos manuscritos (estilo MNIST) · scikit-learn

Plan de la sesión (≈45 min)

Qué veremos

- **A.** Clasificador de margen máximo y el problema dual
- **B.** SVM de margen blando: variables de holgura y el parámetro C
- **C.** Métodos de kernel para clasificación no lineal
- **D.** SVM multiclase y consejos prácticos para conjuntos de datos grandes

Mismo espíritu que la SVR de la sesión 6, pero para predecir una **categoría**: en vez de un tubo alrededor de una curva, buscamos la frontera que separa las clases con el margen más ancho.

De dónde venimos: de SVR a SVM

Módulo 3 (clasificación): lo visto hasta ahora

Sesión	Tema
8	Reg. logística, kNN, métricas básicas
9	Naive Bayes, árboles, ROC/AUC
10 (hoy)	SVM

La conexión con la sesión 6

La SVR buscaba un **tubo** ε-insensible alrededor de una curva. La SVM de clasificación busca el **margen** más ancho entre dos clases. Las ideas de margen blando, C y kernels son las **mismas**.

Siguiente sesión del módulo: redes neuronales para clasificación.

A1 · La idea: la frontera con el margen más ancho

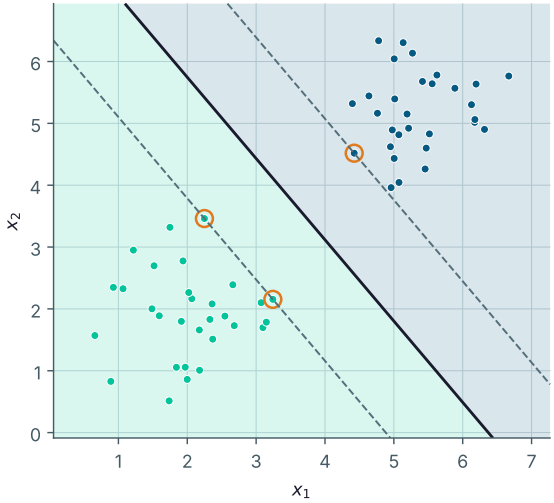
El problema

Con datos separables, hay **infinitas** rectas (o hiperplanos) que separan las dos clases perfectamente. ¿Cuál elegir?

La idea de SVM

Elegir la que deja el **margen** (la distancia a los puntos más cercanos de cada clase) **más ancho**. Un margen amplio generaliza mejor a datos nuevos.

A2 · Vectores de soporte: solo algunos puntos importan



Lo esencial

Solo los puntos **sobre el borde** del margen (marcados en naranja) determinan la frontera: los **vectores de soporte**. Mover cualquier otro punto, lejos del margen, no cambia nada.

En este ejemplo: 3 vectores de soporte definen toda la frontera.

A3 · Formulación: maximizar el margen

El problema primal

Con frontera $w^\top x + b = 0$, el margen es $2/\|w\|$. Maximizarlo equivale a minimizar $\|w\|$:

$$\min_{w,b} \ \tfrac{1}{2}\|w\|^2$$

$$\text{sujeto a } y_i(w^\top x_i + b) \geq 1, \ \forall i$$

Lectura

Las restricciones dicen: cada punto debe quedar **del lado correcto**, a una distancia de al menos 1 (en unidades escaladas) de la frontera.

A4 · El problema dual

Una forma equivalente

El problema primal tiene una versión **dual**, que optimiza multiplicadores $\alpha_i \geq 0$ (uno por cada punto) en vez de w directamente. En la solución, $\alpha_i > 0$ **solo** para los vectores de soporte.

¿Por qué importa?

El dual expresa todo en términos de **productos punto** $x_i^\top x_j$ entre pares de puntos. Eso es justo lo que permite aplicar el **truco del kernel** (sección C).

No es necesario resolver el dual a mano: scikit-learn lo hace internamente. Lo importante es entender **qué** hace posible (vectores de soporte, kernels).

Pausa para pensar (actividad 1, 3 min)

Trabaja de forma individual

1. Si eliminas un punto que **no** es vector de soporte y vuelves a entrenar, ¿cambia la frontera? ¿Y si eliminas uno que sí lo es?
2. ¿Por qué un margen más ancho suele generalizar mejor a datos nuevos?
3. ¿Qué tienen en común el problema dual de SVM y la razón por la que SVR también usaba solo algunos puntos (los que están fuera del tubo ε)?

Respuestas: (1) No cambia si no es vector de soporte; sí puede cambiar, a veces mucho, si lo es. (2) Un margen ancho deja más "espacio de maniobra" ante ruido o puntos nuevos cercanos a la frontera. (3) En ambos casos la solución depende solo de un subconjunto de puntos "informativos" (vectores de soporte), no de todos los datos: por eso SVM y SVR suelen ser modelos *dispersos* (*sparse*).

B1 · ¿Y si los datos no son separables?

El problema real

En la práctica casi nunca hay una frontera que separe perfectamente las clases: hay **traslape**, ruido y casos ambiguos.

La solución

Permitir que **algunos** puntos violen el margen (o incluso queden del lado incorrecto), pero **penalizando** esas violaciones.

B2 · Margen blando: variables de holgura

La formulación

Se añade una holgura $\xi_i \geq 0$ por punto:

$$\min_{w,b,\xi} \ \tfrac{1}{2}\|w\|^2 + C\sum_i \xi_i$$

sujeto a $y_i(w^\top x_i + b) \geq 1 - \xi_i$

Lectura de ξ_i

$\xi_i = 0$: el punto respeta el margen. $0 < \xi_i \leq 1$: está dentro del margen pero del lado correcto. $\xi_i > 1$: está **mal clasificado**.

B3 · El parámetro C: el dial margen–errores

Qué controla

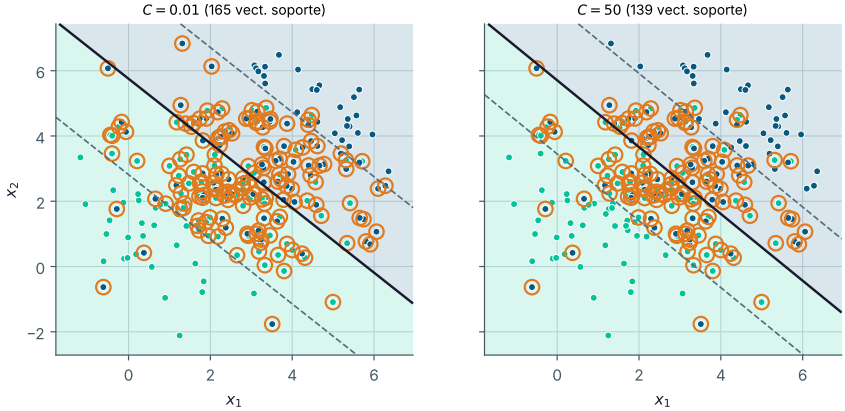
C pondera cuánto **cuesta** cada violación del margen frente a qué tan **ancho** es el margen.

Los dos extremos

C **pequeña**: margen ancho, tolera más violaciones (más sesgo, menos varianza). C **grande**: margen angosto, casi no tolera errores en entrenamiento (menos sesgo, más varianza).

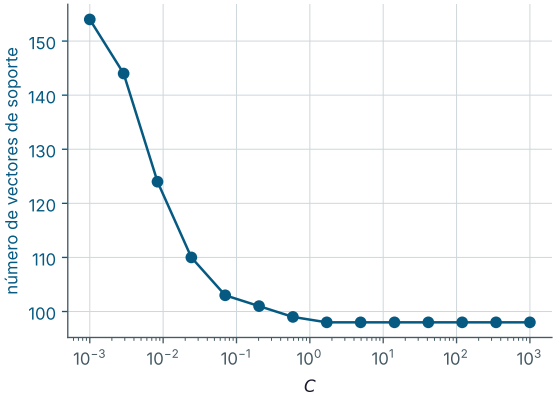
Exactamente el mismo papel que C en la SVR de la sesión 6.

B4 · El efecto de C, en datos



Mismos datos (con algo de traslape y ruido de etiqueta). C chica $\rightarrow$ margen ancho, 165 vectores de soporte. C grande $\rightarrow$ margen angosto, 139 vectores de soporte: el modelo se ajusta más a los datos de entrenamiento.

B5 · Entre más grande C, menos vectores de soporte



Por qué pasa esto

Con C grande, el modelo "prefiere" un margen angosto que solo necesita apoyarse en los puntos más cercanos a la frontera; con C chica, tolera más violaciones y más puntos terminan dentro del margen (y son vectores de soporte).

C se elige con **validación cruzada**, igual que en SVR, buscando el mejor equilibrio sesgo–varianza.

B6 · En scikit-learn: `SVC(kernel="linear")`

```python
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (
    StandardScaler)
from sklearn.model_selection import (
    GridSearchCV)

pipe = Pipeline([
    ("esc", StandardScaler()),
    ("clf", SVC(kernel="linear"))])
malla = {"clf__C": [0.01, 0.1, 1, 10, 100]}
gs = GridSearchCV(pipe, malla, cv=5)
gs.fit(Xtr, ytr)
```

Puntos clave

- SVM es **sensible a la escala**: siempre estandarizar dentro del `Pipeline`.
- `.support_vectors_` da los vectores de soporte encontrados.

Pausa para pensar (actividad 2, 3 min)

Trabaja de forma individual

1. Entrenas una SVM lineal con $C = 1000$ y el modelo acierta 100 % en entrenamiento pero mucho menos en prueba. ¿Qué harías con C?
2. ¿Por qué estandarizar las variables es obligatorio antes de usar SVM, igual que en kNN?
3. ¿Qué le pasaría al margen si $C \to \infty$? ¿A qué caso de la sección A se parece eso?

Respuestas: (1) Bajar C: el modelo está sobreajustando (varianza alta); un margen más ancho debería generalizar mejor. (2) SVM mide distancias y productos punto; una variable con escala mucho mayor dominaría el margen sin estandarizar. (3) El margen se vuelve casi "duro" (casi no tolera violaciones): se parece al caso de margen máximo sin holgura de la sección A, válido solo si los datos son (casi) separables.

C1 · Cuando una recta no alcanza

El problema

Muchos conjuntos de datos no son separables (ni siquiera aproximadamente) con una **recta**, sin importar qué tan grande o chica sea C.

Una idea: transformar los datos

Si los datos no son separables en el espacio original, quizá lo sean en un espacio de **más dimensiones**, construido con combinaciones de las variables originales.

C2 · El truco del kernel, sin tecnicismos

El problema de transformar a mano

Calcular explícitamente esas nuevas variables (a veces infinitas) sería carísimo o imposible.

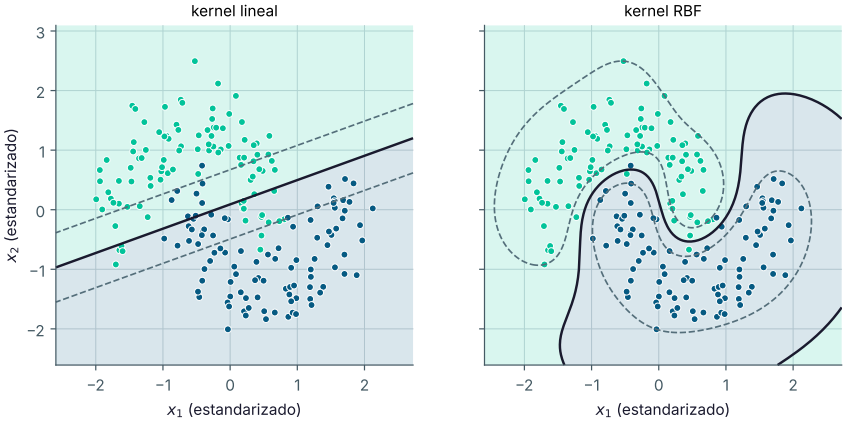
La solución

Recordemos: el dual (sección A4) solo necesita **productos punto** $x_i^\top x_j$. Un **kernel** $k(x_i, x_j)$ calcula directamente el producto punto **como si** los datos ya estuvieran transformados, sin transformarlos nunca.

Por eso se llama "truco": se obtiene el beneficio de un espacio de más dimensiones sin pagar su costo.

Kernel	Fórmula	Cuándo usarlo
Lineal	$k(x, x') = x^\top x'$	datos (casi) separables con una recta
Polinomial	$k(x, x') = (\gamma\, x^\top x' + r)^d$	fronteras curvas “suaves”
RBF (gaussiano)	$k(x, x') = e^{-\gamma \lVert x - x' \rVert^2}$	fronteras muy flexibles; el más usado por defecto

C4 · Kernel lineal vs RBF, en datos



Datos en forma de “lunas” (no separables linealmente). Exactitud en prueba: kernel lineal **0.88**, kernel RBF **0.98**: el kernel RBF captura la curva que la recta no puede.

C5 · El parámetro γ del kernel RBF

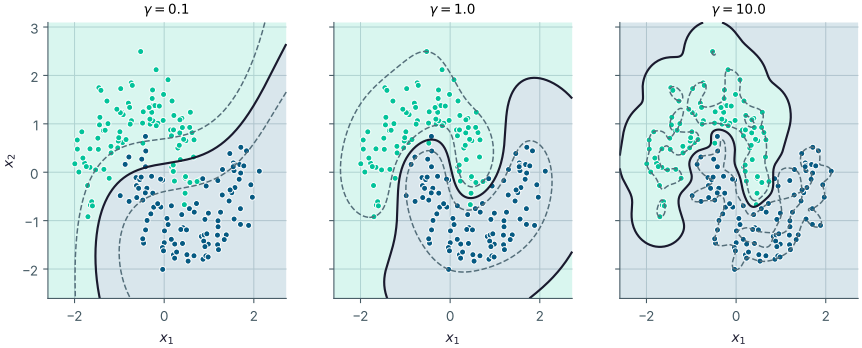
Qué controla

γ controla qué tan "cerca" tiene que estar un punto de otro para influir en él: equivale, intuitivamente, al inverso del alcance de cada punto de soporte.

Los dos extremos

γ **pequeña**: influencia de largo alcance, frontera **suave** (puede no ajustar bien). γ **grande**: influencia muy local, frontera **muy irregular** (riesgo de sobreajuste).

C6 · El efecto de γ, en datos



Mismos datos y mismo C. γ chica apenas se despega de una recta; γ grande memoriza regiones alrededor de puntos individuales: igual que `max_depth` en árboles o k en kNN, γ y C se eligen **juntos** con validación cruzada.

C7 · En scikit-learn: SVC(kernel="rbf")

```python
from sklearn.svm import SVC
import numpy as np

pipe = Pipeline([
    ("esc", StandardScaler()),
    ("clf", SVC(kernel="rbf"))])
malla = {
    "clf__C": np.logspace(-1, 2, 6),
    "clf__gamma": np.logspace(-4, -1, 6)}
gs = GridSearchCV(pipe, malla, cv=5,
    scoring="accuracy")
gs.fit(Xtr, ytr)
```

Puntos clave

- `gamma="scale"` (por defecto) es un buen punto de partida antes de afinar con `GridSearchCV`.
- `kernel="poly"` también acepta el grado `degree`.

Pausa para pensar (actividad 3, 3 min)

Trabaja de forma individual

1. Tus datos forman dos espirales entrelazadas. ¿Qué kernel probarías primero: lineal o RBF? ¿Por qué?
2. Si γ es enorme, ¿qué le pasa a la exactitud en **entrenamiento**? ¿Y a la exactitud en **prueba**?
3. ¿Por qué decimos que C y γ se deben elegir **juntos** y no uno después del otro?

Respuestas: (1) RBF: una frontera lineal no puede separar espirales entrelazadas. (2) En entrenamiento sube (casi memoriza los datos); en prueba típicamente **baja** (sobreajuste). (3) Ambos controlan la flexibilidad del modelo de forma interdependiente: el valor óptimo de γ depende de qué tan estricto sea C, y viceversa; por eso se exploran en una **malla conjunta**.

D1 · De binario a multiclase

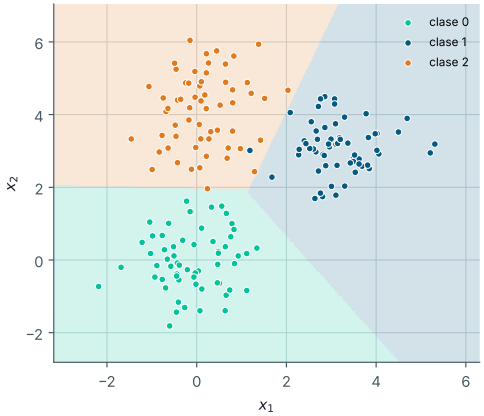
El problema

SVM se define de forma natural para **dos** clases. ¿Cómo se usa con 3 o más?

Mismas estrategias de la sesión 8

Uno contra el resto (OvR): un clasificador binario por clase. **Uno contra uno** (OvO): un clasificador por cada **par** de clases. scikit-learn elige automáticamente (OvO para `SVC`, OvR para `LinearSVC`).

D2 · SVM multiclase, en datos



Con 3 clases

OvR entrena 3 clasificadores binarios ("clase 0 vs. resto", etc.); OvO entrena $\binom{3}{2} = 3$ (en este caso, coinciden en número). Con K clases: OvR entrena K modelos, OvO entrena $\binom{K}{2}$.

Con muchas clases, OvO entrena más modelos pero cada uno sobre menos datos (más rápido por modelo).

D3 · Consejos prácticos para conjuntos de datos grandes

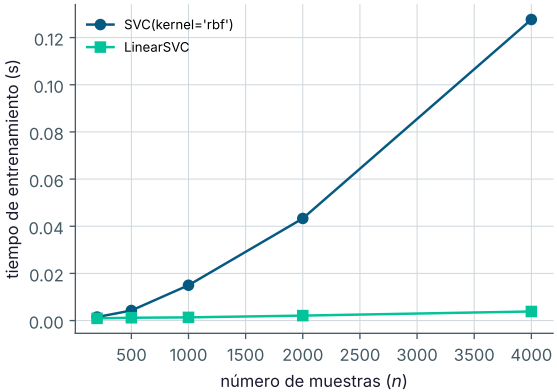
El problema de escalar

`SVC` con kernel no lineal escala mal: el entrenamiento crece aproximadamente **cuadrático o peor** con el número de muestras.

Alternativas

- `LinearSVC`: optimizado para kernel lineal, mucho más rápido con muchas muestras.
- `SGDClassifier(loss="hinge")`: entrena por lotes, escala a millones de filas.
- Aproximaciones de kernel (p. ej. `Nystroem`) + un modelo lineal.

D4 · Tiempo de entrenamiento: `SVC(rbf)` vs `LinearSVC`



Lectura

Con pocas muestras, la diferencia es pequeña. Al crecer n, `SVC` con kernel RBF se vuelve notablemente más lento que `LinearSVC`.

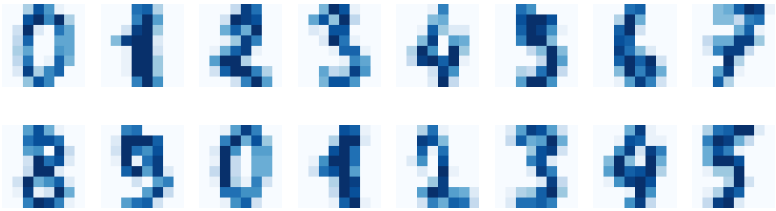
Regla práctica: con decenas de miles de filas o más, empezar con `LinearSVC` o `SGDClassifier`, y usar kernel RBF solo si el desempeño lineal no basta.

D5 · Checklist práctico

Antes de entrenar una SVM

- **Siempre** estandarizar dentro de un `Pipeline` (SVM depende de distancias).
- Empezar con kernel **lineal** como referencia; probar RBF si no basta.
- Elegir C (y γ, si aplica) con `GridSearchCV` y validación cruzada, **juntos**.
- Con datasets grandes, considerar `LinearSVC` o `SGDClassifier` antes que `SVC(kernel="rbf")`.
- Evaluar en prueba **una sola vez**, al final, igual que en todos los modelos del curso.

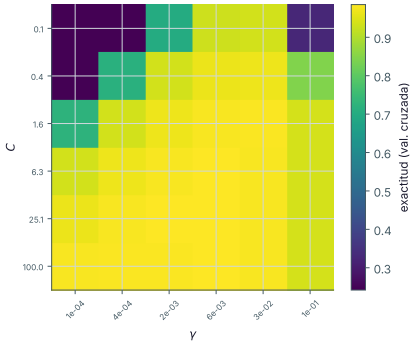
E1 · Vista previa: dígitos manuscritos (estilo MNIST)

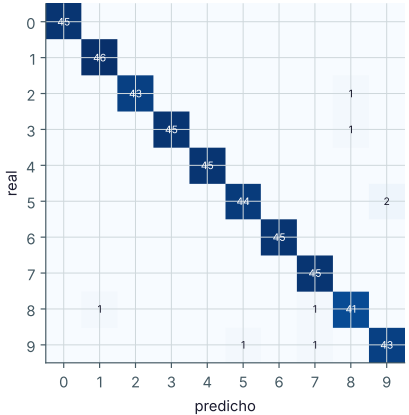


`sklearn.datasets.load_digits`:

1 797 imágenes de 8×8 píxeles, dígitos 0–9. Partición usada en el ejemplo: **1 347** de entrenamiento, **450** de prueba.

E2 · `GridSearchCV` y resultado final, en dígitos





Mejor combinación: $C \approx 6.31$, $\gamma \approx 0.0063$ (exactitud de validación cruzada 0.985). Exactitud en prueba: kernel RBF **0.98**, kernel lineal **0.95**: el kernel RBF gana, pero por poco margen en este conjunto.

Resumen de la sesión

Ideas clave

- **Margen máximo**: la frontera que más se aleja de los puntos de ambas clases; solo los **vectores de soporte** importan.
- **Margen blando**: C equilibra margen ancho contra errores de entrenamiento.
- **Kernels** (lineal, polinomial, RBF) permiten fronteras no lineales sin transformar los datos a mano.

Para llevarse

- C y γ se eligen **juntos**, con validación cruzada.
- OvR/OvO extienden SVM a multiclase, igual que en regresión logística.
- Con datos grandes, preferir `LinearSVC`/`SGDClassifier` sobre `SVC(kernel="rbf")`.

Siguiente sesión

Módulo 3: redes neuronales para clasificación (sesión 11).

Actividad de laboratorio (individual)

SVM con kernel RBF para reconocimiento de dígitos

Cuaderno de Jupyter con un subconjunto de dígitos manuscritos (estilo MNIST, imágenes de 8×8 píxeles, vía `sklearn.datasets.load_digits`). Trabaja **de forma individual**.

1. Explora el conjunto: visualiza algunos dígitos de cada clase (10 clases, dígitos 0–9).
2. Divide en entrenamiento y prueba (estratificado); construye un `Pipeline` con `StandardScaler` + `SVC(kernel="linear")` y mide su exactitud.
3. Repite con `SVC(kernel="rbf")` y compara: ¿mejora el resultado?
4. Usa `GridSearchCV` para elegir C y γ del kernel RBF; visualiza el mapa de calor de exactitud sobre la malla.
5. Evalúa el modelo final en el conjunto de **prueba** (una sola vez) con su matriz de confusión.
6. Identifica qué dígitos se confunden más entre sí, y da una explicación razonable (¿se parecen visualmente?).

Entregable: el cuaderno con el mapa de calor de `GridSearchCV`, la matriz de confusión final y tu interpretación de las confusiones más comunes.



¿Preguntas?

¡Pasemos a la actividad: SVM para reconocimiento de dígitos!
